



GyaanVault

SPPU M.Sc. Data Science — Free Study Material

Lab: Computational Mathematics

DS-505-MJP

Solved Practical Manual

Semester I · Lab (MJP)

This solved practical manual covers the Computational Mathematics lab assignments from the official SPPU M.Sc. Data Science syllabus. Each practical states the aim followed by a complete, runnable solution and the expected output. Use it as a reference — type the code yourself and experiment with the parameters to learn effectively.

Requirements: Python 3.x with numpy, pandas, matplotlib, scikit-learn, scipy installed (pip install numpy pandas matplotlib scikit-learn scipy).

Practical 1: Application of vector spaces

Aim: Test linear independence and find a basis / rank using NumPy.

```
import numpy as np
A = np.array([[1,2,3],[2,4,6],[1,0,1]])
print("Rank:", np.linalg.matrix_rank(A))    # rank < 3 => rows dependent
# Basis of column space via independent columns
_, r = np.linalg.qr(A)
print("Upper-triangular R (pivots show independence):\n", np.round(r,2))
```

Practical 2: Eigenvalues, eigenvectors & inner-product spaces

```
import numpy as np
A = np.array([[4, 1],[2, 3]])
vals, vecs = np.linalg.eig(A)
print("Eigenvalues :", np.round(vals,3))
print("Eigenvectors:\n", np.round(vecs,3))
u, v = np.array([1,2,3]), np.array([4,5,6])
print("Inner product <u,v>:", np.dot(u, v))
print("Norm ||u||      :", np.linalg.norm(u))
```

Practical 3: Linear maps & matrix representation

```
import numpy as np
# Linear map T(x) = A x ; verify T(x+y) = T(x)+T(y)
A = np.array([[2,0],[1,3]])
x, y = np.array([1,1]), np.array([2,-1])
print("T(x+y):", A @ (x+y))
print("T(x)+T(y):", A @ x + A @ y)    # must be equal
```

Practical 4: Basic matrix methods — solving $Ax = b$, LU / QR / SVD

```
import numpy as np
A = np.array([[3,2,-1],[2,-2,4],[-1,0.5,-1]])
b = np.array([1,-2,0])
print("Solution x:", np.linalg.solve(A, b))

U, S, Vt = np.linalg.svd(A)
print("Singular values:", np.round(S,3))
```

Practical 5: Mini case — Mathematics applied to Data Science (SVD image / text)

Aim: Use SVD for dimensionality reduction (low-rank approximation of a matrix / image).

```
import numpy as np
# Low-rank approximation of a data matrix using top-k singular values
M = np.random.rand(8, 6)
U, S, Vt = np.linalg.svd(M, full_matrices=False)
k = 2
M_k = (U[:, :k] * S[:k]) @ Vt[:k, :]
print("Original rank:", np.linalg.matrix_rank(M))
print("Approx rank :", np.linalg.matrix_rank(M_k))
print("Reconstruction error:", np.round(np.linalg.norm(M-M_k),4))
```

This is the core idea behind Latent Semantic Indexing (text mining) and image compression covered in Unit 5.

